Comentario del Revisor

Hola!

Soy Juan Manuel Romero, pero siéntete libre de llamarme Juanma. Soy code reviewer en Tripleten y hoy estaré revisando tu entrega.

Para simular la dinámica de un ambiente de trabajo, si veo algún error, en primer instancia solo los señalaré, dándote la oportunidad de encontrarlos y corregirlos por tu cuenta. En un trabajo real, el líder de tu equipo hará una dinámica similar. En caso de que no puedas resolver la tarea, te daré una información más precisa en la próxima revisión.

Solo un aviso rápido: cuando estés revisando el proyecto, por favor deja mis comentarios originales tal como están. De esta manera, podemos seguir fácilmente el progreso y asegurarnos de que no se nos pase nada por alto. Y, si realizas algún cambio basado en mis comentarios, sería genial si pudieras resaltar esas actualizaciones para que se destaquen.

Puedes encontrar mis comentarios en cajas verdes, amarillas o rojas como estas:

Comentario del Revisor

Éxito. Todo se ha hecho correctamente.

Comentario del Revisor

Observaciones. Algunas recomendaciones.

Comentario del Revisor

Requiere corrección. El bloque requiere algunas correcciones. El trabajo no puede ser aceptado con los comentarios en rojo.

Puedes responderme usando esto:

Respuesta del estudiante.

```
Comienza a programar o generar con IA.
```

Comentario del Revisor #1

Felicidades Irak! Tu trabajo es excelente.

Has completado todos los items necesarios para aprobar la entrega.

Sigue así!

Introducción

Como analista de datos, tu objetivo es **evaluar cómo la movilidad urbana se relaciona con la productividad económica en las principales ciudades latinoamericanas**. Para ello trabajarás con datos reales de TomTom Traffic Index y OECD Cities, que deberás limpiar, combinar y analizar para identificar en qué ciudades conviene invertir en infraestructura de transporte.

Haz doble clic (o ingresa) para editar

🧩 Paso 1: Cargar y explorar

Antes de limpiar o combinar los datos, es necesario **familiarizarte con la estructura de ambos datasets**. En esta etapa, validarás que los archivos se carguen correctamente, conocerás sus columnas y tipos de datos, y detectarás posibles inconsistencias.

1.1 Carga de datos y vista rápida

🎯 **Objetivo:** Importar las librerías necesarias, cargar los archivos CSV en DataFrames y realizar una revisión preliminar para entender su contenido.

Instrucciones:

- Importa las librerías `pandas`, `numpy`, `seaborn` y `matplotlib.pyplot`.
- Carga los archivos usando `pd.read_csv()`:
  - `'/datasets/tomtom_traffic.csv'`
  - `/datasets/oecd_city_economy.csv` `.
- Guarda los DataFrames en las variables `traffic` y `eco`.
- Muestra las primeras 5 filas de cada DataFrame.

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt # importar librerías
```

Comentario del Revisor #1

Buen trabajo al separar la importación de bibliotecas de la carga de los datasets! Esto mejora la estructura y claridad de tu código.

```
# cargar archivos
traffic = pd.read_csv('/datasets/tomtom_traffic.csv')
eco = pd.read_csv('/datasets/oecd_city_economy.csv')#completa el código
```

```
traffic.head(5)# mostrar las primeras 5 filas de traffic
```

	Country	City	UpdateTimeUTC	JamsDelay	TrafficIndexLive	JamsLengthInKms	JamsCount	TrafficIndexWeekAgo	UpdateTimeUTCWeek
0	ARE	abu-dhabi	2025-01-13 04:01:30.001	650.7	36.0	109.1	162.0	30.0	2025-01 04:01:30.
1	ARE	abu-dhabi	2025-01-13 03:46:00.000	540.4	30.0	101.4	136.0	27.0	2025-01 03:46:30.
2	ARE	abu-dhabi	2025-01-13 02:46:30.000	71.8	7.0	18.9	23.0	6.0	2025-01 02:46:30.

```
eco.head(5)# mostrar las primeras 5 filas de eco
```

	Year	City	Country	City GDP/capita	Unemployment %	PM2.5 (µg/m³)	Population (M)
0	2023	buenos-aires	Argentina	15.782,00	6.2%	15,2	15,30
1	2023	sao-paulo	Brazil	14.475,00	9.1%	29,50	22,50
2	2023	rio-de-janeiro	Brazil	13.142,00	9.8%	19,10	13,60
3	2023	brasilia	Brazil	15.999,00	8.3%	13,50	4,70
4	2023	salvador	Brazil	8.761,00	13.1%	16,00	3,90

Tip: Si no usas `print()` la tabla se vera mejor.

🧩 Paso 2: Explorar, limpiar y preparar los datos

Antes de combinar los datasets, inspecciona su estructura, tipos de datos, columnas y valores faltantes. Anota las columnas que necesiten limpieza y luego estandariza los nombres de columnas.

2.1 Explorar la estructura y tipos de datos

🎯 **Objetivo:** Identificar columnas con tipos incorrectos, distribución y nulos, anotar las columnas que requieren conversión.

Instrucciones:

- Usa `.info()` para conocer la estructura de ambos DataFrames.
- Muestra los primeros 3 renglones de cada DF.
- Identifica si los detalles de cada DF estan bien o si requieren correcciones y escribe tus conclusiones en el bloque Markdown.
    - ¿Hay columnas que requieren conversión?¿ Cuáles son? ¿Que tipo de dato ienen y cuál deberían de tener?
    - ¿Hay datos ausentes en alguna columna?

```
traffic.info()# Examinar la estructura de traffic
traffic.head(3)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1004464 entries, 0 to 1004463
Data columns (total 12 columns):
 #   Column                        Non-Null Count    Dtype
---  ------                        --------------    -----
 0   Country                       1004464 non-null  object
 1   City                          1004464 non-null  object
 2   UpdateTimeUTC                 1004464 non-null  object
 3   JamsDelay                     1004464 non-null  float64
 4   TrafficIndexLive              1004464 non-null  float64
 5   JamsLengthInKms               1004464 non-null  float64
 6   JamsCount                     1004464 non-null  float64
 7   TrafficIndexWeekAgo           1004464 non-null  float64
 8   UpdateTimeUTCWeekAgo          1004464 non-null  object
 9   TravelTimeLivePer10KmsMins    1004464 non-null  float64
 10  TravelTimeHistoricPer10KmsMins 1004464 non-null  float64
 11  MinsDelay                     1004464 non-null  float64
dtypes: float64(8), object(4)
memory usage: 92.0+ MB
```

	Country	City	UpdateTimeUTC	JamsDelay	TrafficIndexLive	JamsLengthInKms	JamsCount	TrafficIndexWeekAgo	UpdateTimeUTCWeek
0	ARE	abu-dhabi	2025-01-13 04:01:30.001	650.7	36.0	109.1	162.0	30.0	2025-01 04:01:30.
1	ARE	abu-dhabi	2025-01-13 03:46:00.000	540.4	30.0	101.4	136.0	27.0	2025-01 03:46:30.

En la estructura del DF traffic, se observa que:

- Las columnas UpdateTimeUTC y UpdateTimeUTCWeekAgo son de tipo object, pero representan fechas y horas, por lo que deberían convertirse al tipo datetime para facilitar el análisis temporal y permitir extraer variables como el año.
- Respecto a los valores faltantes, no se observan datos ausentes en ninguna columna, ya que todas muestran 1,004,464 valores no nulos.

```
eco.info()# Examinar la estructura de eco
eco.head(3)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 7 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Year             30 non-null     int64
 1   City             30 non-null     object
 2   Country          30 non-null     object
 3   City GDP/capita  30 non-null     object
 4   Unemployment %   30 non-null     object
 5   PM2.5 (µg/m³)    30 non-null     object
 6   Population (M)   30 non-null     object
dtypes: int64(1), object(6)
memory usage: 1.8+ KB
```

	Year	City	Country	City GDP/capita	Unemployment %	PM2.5 (µg/m³)	Population (M)
0	2023	buenos-aires	Argentina	15.782,00	6.2%	15,2	15,30
1	2023	sao-paulo	Brazil	14.475,00	9.1%	29,50	22,50
2	2023	rio-de-janeiro	Brazil	13.142,00	9.8%	19,10	13,60

Comentario del Revisor #1

Muy buen uso de métodos como *info* y *head* para la exploración de datos. Esto te permitió identificar rápidamente la estructura y posibles problemas de los datos.

En la estructura del DF eco, se observa que:

- Las columnas City GDP/capita, Unemployment %, PM2.5 (µg/m³) y Population (M) están almacenadas como tipo object, aunque representan valores numéricos. Esto probablemente se debe a que contienen símbolos o formatos especiales, como separadores de miles, comas como decimales o el símbolo de porcentaje. Por lo tanto, estas columnas deberán limpiarse y convertirse a tipo float para permitir su análisis numérico.
- En cuanto a los valores faltantes, no se observan datos ausentes, ya que todas las columnas presentan 30 valores no nulos.

2.2 Renombrar columnas

🎯 **Objetivo:** Estandarizar los nombres de columnas para evitar errores y facilitar la unión de los datasets.

Instrucciones:

- Cambia los nombres de las columnas para que tengan el formato `snake_case`.
  - `Country` → `country`
  - `UpdateTimeUTC` → `update_time_utc`
- Verifica que los cambios se hayan aplicado correctamente usando `.columns`.

```
# Estandarizar los nombres de las columnas de traffic
#tu código aquí

traffic = traffic.rename(columns={
    'Country': 'country',
    'UpdateTimeUTC': 'update_time_utc',
    'UpdateTimeUTCWeekAgo': 'update_time_utc_week_ago'
})
# verificar cambios
traffic.columns
```

```
Index(['country', 'City', 'update_time_utc', 'JamsDelay', 'TrafficIndexLive',
       'JamsLengthInKms', 'JamsCount', 'TrafficIndexWeekAgo',
       'update_time_utc_week_ago', 'TravelTimeLivePer10KmsMins',
       'TravelTimeHistoricPer10KmsMins', 'MinsDelay'],
      dtype='object')
```

```
# Estandarizar los nombres de las columnas de eco
#tu código aquí
eco = eco.rename(columns={
    'Country': 'country'
})
# verificar cambios
eco.columns
```

```
Index(['Year', 'City', 'country', 'City GDP/capita', 'Unemployment %',
       'PM2.5 (µg/m³)', 'Population (M)'],
      dtype='object')
```

Comentario del Revisor #1

No realizaste la estandarización de *todos* los nombres de columna a formato *snake_case* en los datasets. Este paso es importante porque mejora la legibilidad del código, reduce errores al manipular datos y favorece la consistencia en todo el pipeline analítico.

Además, recuerda incorporar la lógica en una función reutilizable para asegurar coherencia en futuros proyectos.

2.3 Corregir formatos numéricos y de fecha

🎯 **Objetivo:** Asegurar que las columnas de fechas y valores numéricos estén en formatos correctos para permitir análisis, cálculos y comparaciones precisas.

Instrucciones:

- Convierte las columnas de fecha de `traffic` a formato `datetime`. Haz el cambio a prueba de errores.
- En el dataset `eco`, limpia los valores numéricos:
  - En `city_gdp_capita`: elimina separadores de miles (`.`) y reemplaza las comas (`','`) por puntos (`'.'`) antes de convertir a tipo `float`.
  - En `unemployment_pct`: elimina el símbolo de porcentaje (`%`) y reemplaza las comas (`','`) por puntos (`'.'`) antes de convertir a tipo `float`.
  - En `population_m`: reemplaza las comas (`','`) por puntos (`'.'`) antes de convertir a tipo `float`.
- Finalmente, crea una nueva columna llamada `population` multiplicando `population_m` por 1,000,000 para obtener la población total.

▶ Haz clic para ver la pista

```
# Convertir las columnas de traffic a tipo fecha con pd.to_datetime()
```

```
traffic['update_time_utc'] = pd.to_datetime(traffic['update_time_utc'],errors='coerce',utc=True)#tu código aquí
traffic['update_time_utc_week_ago'] = pd.to_datetime(traffic['update_time_utc_week_ago'],errors='coerce',utc=True)#tu código ac

# verificar el cambio
traffic.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1004464 entries, 0 to 1004463
Data columns (total 12 columns):
 #   Column                          Non-Null Count    Dtype
---  ------                          --------------    -----
 0   country                         1004464 non-null  object
 1   City                            1004464 non-null  object
 2   update_time_utc                 1004464 non-null  datetime64[ns, UTC]
 3   JamsDelay                       1004464 non-null  float64
 4   TrafficIndexLive                1004464 non-null  float64
 5   JamsLengthInKms                 1004464 non-null  float64
 6   JamsCount                       1004464 non-null  float64
 7   TrafficIndexWeekAgo             1004464 non-null  float64
 8   update_time_utc_week_ago        1004464 non-null  datetime64[ns, UTC]
 9   TravelTimeLivePer10KmsMins      1004464 non-null  float64
 10  TravelTimeHistoricPer10KmsMins  1004464 non-null  float64
 11  MinsDelay                       1004464 non-null  float64
dtypes: datetime64[ns, UTC](2), float64(8), object(2)
memory usage: 92.0+ MB
```

Comentario del Revisor #1

Realizaste correctamente la conversión de tipos. Este paso es fundamental para asegurar que los tipos de datos sean coherentes con su uso analítico, evitar errores en operaciones posteriores y permitir cálculos fiables.

Comentario del Revisor #1

Como buena práctica, considera centralizar estas conversiones en un módulo o función de preprocesamiento para mantener un pipeline más robusto y reproducible.

```
# Limpia separadores y convierte columnas numéricas en eco
eco['City GDP/capita'] = eco['City GDP/capita'].astype(str) \
    .str.replace('.', '', regex=False) \
    .str.replace(',', '.', regex=False) \
    .astype(float)

# Limpia porcentaje de desempleo
eco['Unemployment %'] = eco['Unemployment %'].astype(str) \
    .str.replace('%', '', regex=False) \
    .str.replace(',', '.', regex=False) \
    .astype(float)

# Limpia población en millones
eco['Population (M)'] = eco['Population (M)'].astype(str) \
    .str.replace(',', '.', regex=False) \
    .astype(float)

# Calcula población total
eco['population'] = eco['Population (M)'] * 1000000

# verificar el cambio
eco.info()
eco.head(3)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 8 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Year             30 non-null     int64
 1   City             30 non-null     object
 2   country          30 non-null     object
 3   City GDP/capita  30 non-null     float64
 4   Unemployment %   30 non-null     float64
 5   PM2.5 (µg/m³)    30 non-null     object
 6   Population (M)   30 non-null     float64
 7   population       30 non-null     float64
dtypes: float64(4), int64(1), object(3)
memory usage: 2.0+ KB
```

	Year	City	country	City GDP/capita	Unemployment %	PM2.5 (µg/m³)	Population (M)	population
0	2023	buenos-aires	Argentina	15782.0	6.2	15,2	15.3	15300000.0
1	2023	sao-paulo	Brazil	14475.0	9.1	29,50	22.5	22500000.0
2	2023	rio-de-janeiro	Brazil	13142.0	9.8	19,10	13.6	13600000.0

Comentario del Revisor #1

Eliminaste correctamente los símbolos de porcentaje, comas y puntos en las columnas numéricas. Esta limpieza es fundamental para evitar errores en conversiones de tipo y garantizar que las operaciones matemáticas se ejecuten sin problemas.

Creaste correctamente la columna *population* a partir de *population_m* multiplicando por 1 000 000. Esta transformación asegura que la variable quede expresada en unidades estándar y facilita comparaciones y cálculos posteriores.

Comentario del Revisor #1

Como buena práctica, procura documentar estas derivaciones.

Finalmente, validas los datos nuevamente luego de los cambios. Esto permite validar que los tipos de datos, estadísticas básicas y posibles inconsistencias estén alineadas con lo esperado tras las transformaciones.

🧩 Paso 3: Extraer año y filtrar

Extraer el año permite filtrar la información y trabajar solo con el período más reciente y relevante.

3.1 Extraer columna año y filtrar 2024

🎯 **Objetivo** Identificar el año de cada registro y mantener solo los registros del 2024.

Intrucciones

- Como el DataFrame `traffic` no tiene una columna de año, utiliza el atributo `.dt.year` sobre su columna de fecha para crear una nueva columna llamada `year`.
- Filtra las filas donde el año sea **2024**.
- Utiliza `.copy()` para crear dos nuevos DataFrames (`traffic_2024` y `eco_2024`) para evitar modificar el dataset original.

```
# Extraer el año de las fechas en update_time_utc
# traffic['year'] = ...
traffic['year'] = traffic['update_time_utc'].dt.year
# Verificar cambio
traffic.head(3)
```

	country	City	update_time_utc	JamsDelay	TrafficIndexLive	JamsLengthInKms	JamsCount	TrafficIndexWeekAgo	update_time
0	ARE	abu-dhabi	2025-01-13 04:01:30.001000+00:00	650.7	36.0	109.1	162.0	30.0	2025-01-06
1	ARE	abu-dhabi	2025-01-13 03:46:00+00:00	540.4	30.0	101.4	136.0	27.0	03:46:3
2	ARE	abu-dhabi	2025-01-13 02:46:30+00:00	71.8	7.0	18.9	23.0	6.0	2025-01-06

Comentario del Revisor #1

Extraíste correctamente el año desde la columna *update_time_utc* y lo asignaste a *traffic['year']*. Este paso facilita el análisis temporal y permite realizar agregaciones o comparativas por año de manera eficiente.

```
# Filtra los registros del año 2024

traffic_2024 = traffic[traffic['year'] == 2024].copy()

eco_2024 = eco[eco['Year'] == 2024].copy()

# Revisar dataframes nuevos
display(traffic_2024.head())
display(eco_2024.head())
```

	country	City	update_time_utc	JamsDelay	TrafficIndexLive	JamsLengthInKms	JamsCount	TrafficIndexWeekAgo	update_ti
302	ARE	abu-dhabi	2024-12-31 23:01:30+00:00	12.9	5.0	2.5	5.0	2.0	23:0
303	ARE	abu-dhabi	2024-12-31 22:01:30+00:00	136.0	21.0	20.6	32.0	3.0	2024-12-
304	ARE	abu-dhabi	2024-12-31 21:16:30+00:00	455.2	31.0	40.4	72.0	4.0	2024-12-
305	ARE	abu-dhabi	2024-12-31 20:01:00.001000+00:00	399.4	27.0	38.0	75.0	6.0	20:0
306	ARE	abu-dhabi	2024-12-31 19:46:00+00:00	366.4	28.0	39.8	82.0	9.0	2024-12-

	Year	City	country	City GDP/capita	Unemployment %	PM2.5 (µg/m³)	Population (M)	population
15	2024	buenos-aires	Argentina	18117.0	7.2	14,50	15.4	15400000.0
16	2024	sao-paulo	Brazil	14703.0	8.5	28,00	22.6	22600000.0
17	2024	rio-de-janeiro	Brazil	13349.0	9.2	18,40	13.7	13700000.0
18	2024	brasilia	Brazil	16251.0	7.8	12,80	4.8	4800000.0
19	2024	salvador	Brazil	8899.0	12.4	15,20	3.9	3900000.0

Comentario del Revisor #1

Filtraste correctamente ambos DataFrames y utilizaste *.copy()*, lo cual es una buena práctica para evitar advertencias de *SettingWithCopy* y asegurar que las modificaciones posteriores no afecten al DataFrame original. Mantener este hábito ayuda a trabajar con objetos independientes y garantiza un flujo de datos más seguro y controlado.

🧩 Paso 4: Analizar y resumir datos de movilidad

Como el dataset de tráfico contiene **múltiples registros por ciudad**. En esta parte, calcularás los promedios anuales por ciudad para simplificar el análisis y obtener una visión más clara de las tendencias generales.

4.1 Calcular promedios de tráfico por ciudad

🎯 **Objetivo:** Obtener una vista consolidada del tráfico promedio por ciudad y año, para analizar patrones generales sin depender de datos diarios.

Instrucciones

- Agrupa los datos por `city`, `country` y `year`.
- Calcula el promedio **solo de las métricas de tráfico más relevantes**: como `jams_delay`, `traffic_index_live`, `jams_length_kms`, `jams_count`, `mins_delay`, y tiempos de viaje (`travel_time_live_per_10kms_mins` y `travel_time_hist_per_10kms_mins`).
- Guarda el resultado como `traffic_city_year_2024`, mantén las columnas como variables (no índices).

▶ Haz clic para ver la pista

```
traffic_city_year_2024 = traffic_2024.groupby(
    ["City","country","year"]
)[[
    "JamsDelay",
    "TrafficIndexLive",
    "JamsLengthInKms",
    "JamsCount",
    "MinsDelay",
```

```
    "TravelTimeLivePer10KmsMins",
    "TravelTimeHistoricPer10KmsMins"
]].mean().reset_index()

traffic_city_year_2024.head()
```

	City	country	year	JamsDelay	TrafficIndexLive	JamsLengthInKms	JamsCount	MinsDelay	TravelTimeLivePer10KmsMins	Travel
0	a-coruna	ESP	2024	17.935187	15.259774	2.198002	4.934405	0.774172	16.267977	
1	aachen	DEU	2024	26.732141	20.960314	3.892586	6.601832	0.792968	13.397861	
2	aarhus	DNK	2024	21.200616	16.575891	2.736736	6.109987	0.495276	15.219292	
3	abu-dhabi	ARE	2024	171.157315	13.902028	24.507380	47.268019	0.139764	9.829092	

Comentario del Revisor #1

Agrupaste correctamente por *city, country* y *year* utilizando *.groupby(...).*

Comentario del Revisor #1

Nota que podrías utilizar *.groupby(..., as_index=False)* para obtener fácilmente la agregación que buscas desde un principio.

Calculaste correctamente la media de las métricas relevantes. Esta agregación permite obtener una representación más estable del comportamiento general y es un paso adecuado dentro del proceso de análisis descriptivo.

🧠 Momento de reflexión

¡Excelente trabajo hasta aquí!

Ahora que ya tienes los promedios anuales por ciudad, es momento de **observarlos** con atención.

Piensa:

- ¿Cuál crees que tiene el mayor tiempo promedio de tráfico?
- ¿Será una ciudad de **Europa**, de **Latinoamérica** o de **otra región** del mundo?

Para descubrirlo, ejecuta esta línea de código:

`traffic_city_year_2024.sort_values(["jams_delay"], ascending=False)`

🔍 Observa qué ciudad aparece en los primeros lugares.

¿Te sorprenden los resultados? , ¿Coinciden con lo que imaginabas?

```
# tu código aquí
traffic_city_year_2024.sort_values(["JamsDelay"], ascending=False)
```

	City	country	year	JamsDelay	TrafficIndexLive	JamsLengthInKms	JamsCount	MinsDelay	TravelTimeLivePer10KmsMins	Tr
221	mexico-city	MEX	2024	2833.057892	34.218190	389.239265	594.969392	1.855542	21.809092	
352	tokyo	JPN	2024	2152.574357	36.805059	373.069734	518.809420	0.698152	22.443778	
246	new-york	USA	2024	2133.400000	28.210388	398.227892	544.474902	1.396351	18.505043	
200	london	GBR	2024	2050.703662	29.230166	287.632868	471.795554	1.325160	17.714139	
211	manila	PHL	2024	1741.493381	66.129402	246.858082	341.881205	2.469894	27.134629	
...	...	...	...	...	...	...	...	...	...	
111	dunedin	NZL	2024	4.651175	15.430809	0.712315	1.591384	0.633294	16.226009	
363	uppsala	SWE	2024	4.194486	13.939168	0.656368	1.349672	0.501802	15.746717	
123	fujairah	ARE	2024	4.025959	10.907719	0.731910	1.373006	0.194951	11.662590	
12	almere	NLD	2024	3.633523	6.290478	0.506362	1.064063	-0.017544	9.467150	
117	el-paso	USA	2024	0.299024	2.340435	0.031114	0.078118	-0.250676	10.690269	

La ciudad con el mayor tiempo promedio de tráfico es... mexico-city

🧩 Paso 5: Unir movilidad y economía

Combinar datasets te permite analizar cómo se relacionan los indicadores económicos con los de movilidad.

5.1 Unir tráfico (tabla principal) con indicadores económicos

🎯 **Objetivo:** Combinar la información de tráfico y economía en un solo DataFrame para analizar cómo las condiciones económicas se relacionan con la movilidad urbana.

Instrucciones

- Selecciona solo las **columnas relevantes** de cada dataset (por ejemplo, variables clave de tráfico y de economía).
- Usa `.copy()` al crear subconjuntos para evitar modificar el dataset original.
- Une ambos DataFrames y define como **claves de unión** a `city` y `year`.
- Mantén solo las ciudades y años presentes en ambos datasets.
- Guarda el resultado en una nueva variable llamada `merged` y muestra las primeras 5 filas.

▶ Haz clic para ver la pista

```
# Seleccionar columnas clave de tráfico y economía
left_cols = ['City','country','year','JamsDelay','TrafficIndexLive',
             'JamsLengthInKms','JamsCount','MinsDelay',
             'TravelTimeLivePer10KmsMins','TravelTimeHistoricPer10KmsMins']

right_cols = ['City','Year','City GDP/capita','Unemployment %','PM2.5 (µg/m³)','Population (M)']


# Usar .copy() para crear datasets reducidos
traffic_2024_small = traffic_city_year_2024[left_cols].copy()

eco_2024_small = eco_2024[right_cols].copy()


# Unir datasets usando ciudad y año
merged = traffic_2024_small.merge(
    eco_2024_small,
    left_on=['City','year'],
    right_on=['City','Year'],
    how='inner'
)


# Mostrar las primeras 5 filas
merged.head()
```

	City	country	year	JamsDelay	TrafficIndexLive	JamsLengthInKms	JamsCount	MinsDelay	TravelTimeLivePer10KmsMins	Tra
0	belo-horizonte	BRA	2024	263.047879	19.428946	44.038129	68.805422	0.487228	18.304538	
1	bogota	COL	2024	1141.552364	37.614273	140.893564	230.566550	1.699628	24.992185	
2	brasilia	BRA	2024	101.576326	11.258220	18.337133	27.280140	0.193442	13.338658	
3	buenos-aires	ARG	2024	571.089593	17.756012	100.287844	137.359860	0.416566	17.907916	
4	curitiba	BRA	2024	183.469274	14.954545	30.050044	46.898164	0.139965	17.258700	

Comentario del Revisor #1

Ejecutaste correctamente la operación de *merge* utilizando `pd.merge(..., on=['city', 'year'], how='inner')`. Esta unión es apropiada cuando se desea trabajar únicamente con las observaciones presentes en ambos DataFrames y mantener consistencia en las claves de análisis.

🧩 Paso 6: Visualización y análisis de relaciones

Ahora que tienes un dataset limpio y unificado, es momento de **visualizar patrones**. Los gráficos te ayudarán a entender cómo se relacionan las variables económicas con las de movilidad urbana.

6.1 Visualizar relaciones entre economía y tráfico

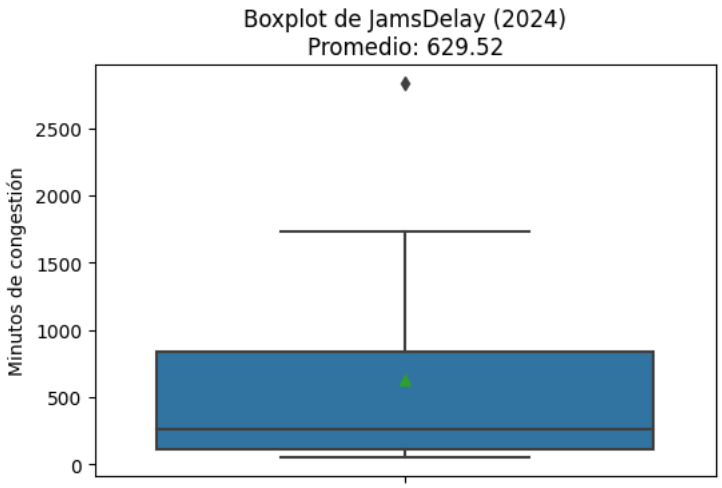
🎯 **Objetivo:** Analizar visualmente la distribución y la relación entre indicadores de tráfico y economía en 2024, para identificar posibles patrones o tendencias generales entre ambas variables.

Instrucciones

- Usa las librerías `seaborn` y `matplotlib.pyplot` para generar los gráficos.
- Visualiza la distribución del **tráfico** (`jams_delay`) mediante:
  - **Boxplot** → para observar la media, mediana y detectar valores atípicos.
- Visualiza la distribución de la **economía** (`city_gdp_capita`) mediante:
  - **Histograma** → para analizar la forma de la distribución y el valor promedio del PIB per cápita.
- Finalmente, **compara ambas variables**, para observar si existe alguna relación entre ellas, haciendo un solo gráfico de barras donde aparezcan ambos indicadores.
- Recuerda agregar título y etiquetas a los ejes de tus gráficos.
- Observa y comenta los patrones, valores extremos o posibles relaciones que identifiques.

Tip: Dentro de los parentesis del boxplot, agrega `showmeans=True` para ver la media en el gráfico.

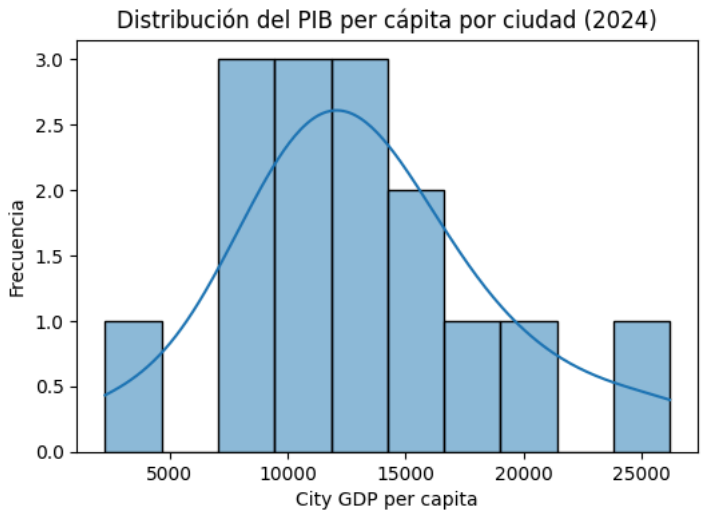
```
# Crear boxplot para observar el comportamiento de los minutos de congestión
plt.figure(figsize=(6,4))

sns.boxplot(data=merged, y='JamsDelay', showmeans=True)

# obtener promedio para mostrarlo en título
mean_value = merged['JamsDelay'].mean()

plt.title(f'Boxplot de JamsDelay (2024)\nPromedio: {mean_value:.2f}')
plt.ylabel('Minutos de congestión')
plt.xlabel('')

plt.show()
```



Comentario del Revisor #1

La visualización es adecuada para comparar la distribución de las variables y detectar outliers, rangos y diferencias entre grupos. Utilizar este tipo de gráfico demuestra un enfoque sólido para el análisis exploratorio.

```
plt.figure(figsize=(6,4))

sns.histplot(data=merged, x='City GDP/capita', bins=10, kde=True)

plt.title('Distribución del PIB per cápita por ciudad (2024)')
plt.xlabel('City GDP per capita')
plt.ylabel('Frecuencia')

plt.show()
```

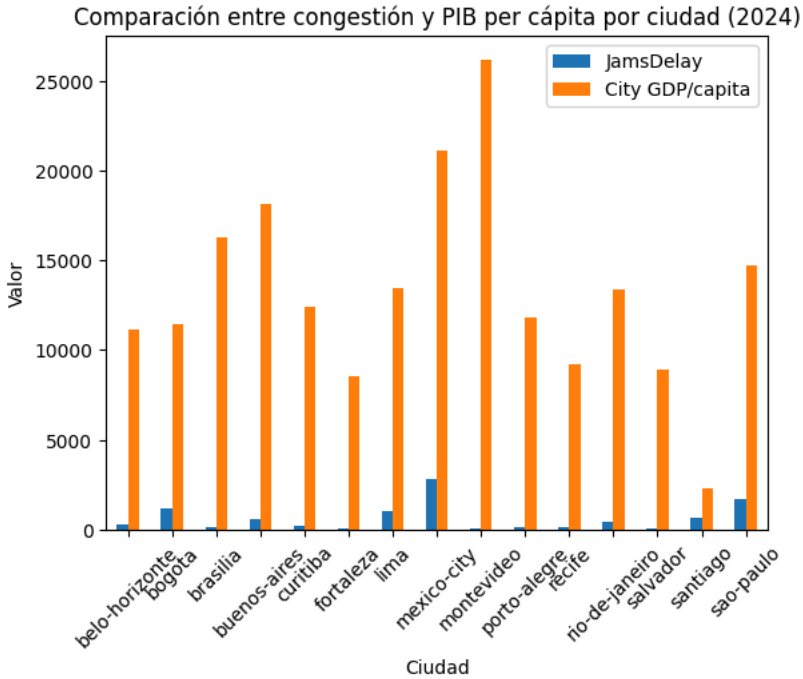


Comentario del Revisor #1

Generaste correctamente el histograma de PIB per cápita y añadiste etiquetas adecuadas. Esta visualización es útil para entender la distribución de la variable, detectar sesgos y evaluar la presencia de valores atípicos.

```
plt.figure(figsize=(10,5))

merged.plot(
    x='City',
    y=['JamsDelay','City GDP/capita'],
    kind='bar'
)

plt.title('Comparación entre congestión y PIB per cápita por ciudad (2024)')
plt.xlabel('Ciudad')
plt.ylabel('Valor')

plt.xticks(rotation=45)

plt.show()
```

```
<Figure size 1000x500 with 0 Axes>
```



Comentario del Revisor #1

Generaste correctamente la comparación entre tráfico y economía utilizando con las variables *jams_delay* y *city_gdp_capita*. Esta visualización permite explorar tendencias y posibles relaciones entre congestión y actividad económica, ofreciendo una perspectiva clara para el análisis exploratorio.

Comentario del Revisor #1

Como buena práctica, recuerda verificar coherencia de escalas.

Tip: Antes del `plt.show()` agrega el código `plt.xticks(rotation=90)` para rotar las etiquetas del eje X en 90 grados.

🧠 Reflexiona

Excelente trabajo llegando a esta etapa del análisis. Antes de avanzar, revisa tus gráficos, tómate un momento para pensar:

- ¿Las ciudades con mayor PIB per cápita también presentan más congestión?
- ¿O sucede lo contrario, o no existe una relación clara?

Escribe tus comentarios:

Al analizar los gráficos generados, no se observa una relación completamente clara entre el PIB per cápita de una ciudad y los niveles de congestión (JamsDelay).

Algunas ciudades con alto PIB per cápita también presentan niveles elevados de congestión, lo cual puede explicarse por una mayor actividad económica, densidad poblacional y volumen de transporte urbano. Sin embargo, también existen ciudades con economías fuertes pero niveles de congestión moderados, posiblemente debido a una mejor infraestructura de transporte, planeación urbana o sistemas de movilidad más eficientes.

Por otro lado, algunas ciudades con PIB per cápita menor también presentan niveles considerables de tráfico, lo que podría estar relacionado con problemas de infraestructura, crecimiento urbano desordenado o sistemas de transporte menos desarrollados.

En general, los gráficos sugieren que no existe una relación directa o lineal entre el PIB per cápita y la congestión urbana, ya que el tráfico parece depender de múltiples factores adicionales como la densidad poblacional, la infraestructura vial y las políticas de movilidad.

Comentario del Revisor #1

Incluiste comentarios claros sobre los patrones y posibles relaciones observadas en los datos.

🧩 Paso 7: Exportar y documentar resultados

En esta etapa final consolidarás todo tu trabajo: guardarás el dataset limpio y crearás un resumen que documente los resultados del proyecto.

7.1 Guardar dataset final

🎯 **Objetivo:** Generar un CSV limpio, reproducible y con columnas relevantes para análisis posterior.

Instrucciones

- Exporta el DataFrame `merged` con el nombre: `ladb_mobility_economy_2024_clean.csv`
- Usa `index=False` para no incluir el índice.

```
# Exporta el dataset final como CSV
merged.to_csv("ladb_mobility_economy_2024_clean.csv", index=False)
```

Comentario del Revisor #1

Exportaste correctamente el DataFrame *merged*.

Comentario del Revisor #1

Te recomiendo verificar la exportación comprobando el tamaño del archivo con `os.path.getsize()`, una práctica adecuada para confirmar que el proceso se ejecutó correctamente y que el archivo no quedó vacío o corrupto. Mantener este tipo de verificaciones mejora la confiabilidad del pipeline de datos.

Para poder ver o descargar el archivo generado:
En el menú lateral que esta a la izquierda, ve hasta la parte de abajo, a la sección de **Exportar dataset** para más información.

✅ Entregables

1. **Notebook** `.ipynb` con todas las celdas (código + comentarios).
2. **CSV final**: `ladb_mobility_economy_2024_clean.csv`.
3. **Resumen ejecutivo breve** en Markdown (3–5 párrafos).

📄 Resumen ejecutivo (plantilla)

> Completa este resumen al finalizar el análisis. Mantén 3–5 párrafos cortos, claros y accionables.

Contexto & objetivo:

- Responde la pregunta central del análisis: ¿qué relación existe entre la movilidad urbana (congestión, tiempos de viaje) y la productividad económica (PIB per cápita)?
- Explica brevemente las variables clave utilizadas y su relevancia para la toma de decisiones.

Cobertura de datos:

- Especifica los años analizados, número de ciudades y países incluidos.

Metodología (alto nivel):

- Describe los procesos principales: limpieza de datos (formatos, estandarización de columnas).
- Explica la agregación por ciudad–año y el uso de una unión INNER para integrar tráfico y economía.
- Menciona las validaciones visuales empleadas (distribuciones, outliers, tendencias generales).

Hallazgos iniciales:

- Resume los patrones más importantes entre índices de tráfico y PIB per cápita.
- Destaca anomalías u outliers que podrían requerir revisión adicional o un análisis más profundo.

Recomendaciones
Aterriza los hallazgos en acciones: ciudades prioritarias, necesidad de validar fuentes, requerimiento de análisis adicionales, o propuestas de inversión.

- ¿Qué ciudad : Bogotá, Lima o Buenos Aires o alguna otra en particular, muestra la mayor correlación significativa entre altos niveles de congestión vehicular y bajos indicadores de productividad económica, sugiriendo ser una ciudad prioritaria para inversión en infraestructura de transporte?

📄 Resumen ejecutivo Contexto & objetivo

El objetivo de este análisis fue explorar la relación entre la movilidad urbana y la productividad económica en distintas ciudades durante 2024. En particular, se buscó identificar si mayores niveles de congestión vehicular, medidos a través de indicadores como JamsDelay y tiempos de viaje, se relacionan con diferencias en la productividad económica representada por el PIB per cápita de la ciudad. Estas variables son relevantes porque la congestión puede impactar la eficiencia económica, los costos logísticos y la calidad de vida urbana.

Cobertura de datos

El análisis se realizó utilizando datos correspondientes al año 2024, integrando información de movilidad urbana proveniente de registros de tráfico y datos económicos provenientes de indicadores urbanos. El dataset final incluye múltiples ciudades de distintos países, donde cada observación representa el promedio anual de indicadores de tráfico por ciudad combinado con métricas económicas y ambientales.

Metodología (alto nivel)

Primero se realizó un proceso de limpieza y estandarización de datos, corrigiendo formatos de fecha, nombres de columnas y tipos de datos para asegurar consistencia entre datasets. Posteriormente se agregaron los datos de tráfico para obtener promedios anuales por ciudad y año, reduciendo la granularidad diaria a una visión más estratégica. Después se integraron los datasets de movilidad y economía mediante una unión INNER utilizando las variables ciudad y año, garantizando que solo se analizaran registros presentes en ambas fuentes. Finalmente se realizaron visualizaciones exploratorias como boxplots, histogramas y gráficos comparativos para identificar distribuciones, posibles valores atípicos y tendencias generales entre las variables.

Hallazgos iniciales

Los resultados sugieren que no existe una relación lineal clara entre el PIB per cápita y los niveles de congestión urbana. Algunas ciudades con altos niveles de productividad económica también presentan niveles elevados de congestión, posiblemente debido a mayor

actividad económica y densidad urbana. Sin embargo, también se identificaron ciudades con niveles moderados de congestión y economías fuertes, lo que podría reflejar una mejor planificación urbana o infraestructura de transporte más eficiente. Asimismo, algunos valores extremos en congestión sugieren posibles diferencias en infraestructura vial o sistemas de transporte público.

Recomendaciones

Con base en los patrones observados, Lima aparece como una ciudad prioritaria para análisis adicional, ya que combina niveles elevados de congestión con indicadores económicos relativamente más bajos en comparación con otras ciudades analizadas. Esto sugiere que la congestión podría estar afectando la eficiencia económica y la movilidad urbana. Se recomienda profundizar el análisis incorporando variables adicionales como densidad poblacional, infraestructura de transporte público y crecimiento urbano, así como validar los datos con fuentes complementarias. Desde una perspectiva de política pública, ciudades con estas características podrían beneficiarse de